ЛР5. DAST и конвейер безопасности в GitHub Actions

Время: 4 академических часа (два занятия): часть А — OWASP ZAP, часть Б — конвейер.
Что получится: ваш проект опубликован на GitHub; на каждое изменение автоматически запускаются тесты, Gitleaks, Semgrep, Trivy и ZAP; уязвимое изменение конвейер не пропускает.

Перед началом: выполнены ЛР1–ЛР4, запущен Docker, есть учётная запись github.com.

Коротко о главном

DAST проверяет работающее приложение снаружи, как посетитель. Код ему не нужен.

	SAST (ЛР2)	SCA (ЛР4)	DAST (ЛР5)
Что проверяет	исходный код	версии библиотек	работающий сайт
Видит только он	опасные конструкции в коде	известные уязвимые версии	заголовки и реальные ответы сервера
Не видит	настройки запуска, права доступа	ваши собственные ошибки	код и страницы, до которых не дошёл

- **Пассивное сканирование (baseline)** — ZAP обходит сайт и анализирует ответы, ничего не атакуя. 1–2 минуты, годится для конвейера.
- **Активное сканирование** — ZAP подставляет вредоносные значения. Только против своих систем: против чужих без разрешения это правонарушение.
- `FAIL: 0` не значит «безопасно». Сканер проверил только то, до чего добрался, и только как анонимный посетитель.

CI в GitHub Actions — файл YAML в `.github/workflows/`, который GitHub выполняет на каждое изменение.

Понятие	Что это
workflow	весь конвейер, один файл YAML
job (задание)	группа шагов на отдельной виртуальной машине; задания идут параллельно
step (шаг)	одна команда
needs	«запускать только после успеха других заданий»
artifact	сохранённый файл: отчёт сканера, SBOM
pull request (PR)	запрос на внесение изменений из отдельной ветки в `main`

Главный механизм: **если команда вернула не ноль — шаг красный, изменение помечено как непрошедшее.** Поэтому в ЛР2–ЛР4 вы учили параметры `--error` и `--exit-code 1`.

Правила безопасности самого конвейера: минимальные права (`permissions: contents: read`), точные версии сканеров, секреты только в GitHub Secrets, сбой сканера — это «проверка не выполнена», а не «всё чисто».

Часть A. OWASP ZAP

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
export MSYS_NO_PATHCONV=1          # только Windows, Git Bash
```

Сокращение для запуска ZAP (действует до закрытия терминала):

```
zap() { docker run --rm --add-host host.docker.internal:host-gateway -v "$PWD/reports:/zap/wrk/:rw" ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://host.docker.internal:5000 "$@"; }
```

`host.docker.internal` — имя, под которым контейнер со сканером видит ваш компьютер.

Шаг A1. Запустить цель

```
docker compose up -d --build --wait
curl http://127.0.0.1:5000/health
curl -sI http://127.0.0.1:5000/
```

Что увидите: `"status":"ok"`, затем заголовки ответа сервера. Запишите их — после исправления сравните.

Шаг A2. Пассивное сканирование

```
zap -r zap-before.html -I
```

- `-r` — сохранить отчёт HTML в `reports`;
- `-I` — не считать предупреждения ошибкой запуска.

Образ ZAP большой (около 2 ГБ) — первая загрузка займёт несколько минут.

Что увидите в последней строке:

```
FAIL-NEW: 0  FAIL-INPROG: 0  WARN-NEW: 10  WARN-INPROG: 0  INFO: 0  IGNORE: 0  PASS: 58
```

Если в Linux отчёт не создаётся (`Permission denied`) — выполните `chmod 777 reports`.

Откройте `reports/zap-before.html` в браузере и заполните таблицу по десяти предупреждениям:

Код	Название	Что это значит простыми словами	Настоящая проблема?

Код	Название	Что это значит простыми словами	Настоящая проблема?
10038	Content Security Policy Header Not Set	сервер не сообщает браузеру, откуда можно загружать сценарии	да
…			

Два вопроса:

1. Какие адреса посетил ZAP? Был ли он на страницах, доступных только после входа?
2. ZAP показал `FAIL: 0`. Сообщил ли он, что любой посетитель может прочитать профиль администратора (атака 3 из ЛР1)?

В отчёт: итоговая строка, таблица, два ответа.

Шаг А3. То, чего не увидел ни один сканер

```
curl -s "http://127.0.0.1:5000/profile?id=1" | grep -E "Email|заметка"
```

Что увидите: почту и закрытую заметку администратора — без входа в систему.

Ни Semgrep, ни Gitleaks, ни Trivy, ни ZAP об этом не сообщили. Сканер не знает, что профиль номер 1 «чужой»: технически запрос корректен. Нарушено правило «кому что можно видеть», а оно есть только у людей. Такие дефекты ищут вручную и закрепляют тестами.

В отчёт: команда и вывод.

Шаг А4. Применить исправление

```
cp -r ../secure-development-labs/solutions/lab5/. .
git diff --stat
pytest -q
```

Что увидите: изменены три файла, `11 passed`.

Что изменилось:

Изменение	Результат
`/profile` проверяет, кто спрашивает	без входа — код 401; чужой профиль — код 403; администратору можно
сервер добавляет заголовки безопасности	`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Permissions-Policy`
новые тесты	проверяют 401, 403 и наличие заголовков

Новые тесты — способ объяснить конвейеру правило, которого не знал сканер. Посмотрите их названия:

```
pytest -v | grep -E "profile|headers"
```

Шаг A5. Повторное сканирование

```
docker compose up -d --build --wait
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:5000/profile?id=1"
curl -sI http://127.0.0.1:5000/
zap -r zap-after.html -I
```

Что увидите: 401; четыре новых заголовка; ZAP — `WARN-NEW: 6` вместо 10.

Примите решение по каждому из шести оставшихся предупреждений:

Код	Название	Решение	Обоснование
10202	Absence of Anti-CSRF Tokens	настоящая проблема, в план доработок	формы не защищены от подделки запросов
10036	Server Leaks Version Information		
10031	User Controllable HTML Element Attribute		
90004	Cross-Origin-Embedder-Policy Header Missing		
10049	Non-Storable Content		
10111	Authentication Request Identified		

Варианты решения: «в план доработок», «информационное, действий не требует», «ложное срабатывание».

В отчёт: сравнение 10 → 6, таблица решений.

Шаг A6. Превратить решения в правила сканера

Файл правил говорит ZAP, что считать провалом. Создайте его командой (колонки разделяются табуляцией, вручную легко ошибиться):

```
printf '10038\tFAIL\t(Content-Security-Policy)\n10020\tFAIL\t(Anti-clickjacking)
\n10021\tFAIL\t(X-Content-Type-Options)\n10202\tWARN\t(Anti-CSRF: backlog)\n1004
9\tIGNORE\t(informational)\n10111\tIGNORE\t(informational)\n' > zap-rules.tsv
cat zap-rules.tsv
```

Правило	Действие	Смысл
10038, 10020, 10021	FAIL	пропал заголовок безопасности — остановить конвейер
10202	WARN	известная проблема в плане доработок
10049, 10111	IGNORE	информационные сообщения

Запустите с правилами и посмотрите код завершения:

```
cp zap-rules.tsv reports/
zap -c zap-rules.tsv -r zap-rules-check.html -I ; echo "Код: $?"
```

Что увидите: `FAIL-NEW: 0`, `IGNORE: 2`, `Код: 0`.

Теперь проверьте, что порог срабатывает. Временно объявите провалом отсутствие защиты от CSRF:

```
printf '10202\tFAIL\t(Anti-CSRF)\n' > reports/zap-rules.tsv
zap -c zap-rules.tsv -I ; echo "Код: $?"
```

Что увидите: `FAIL-NEW: 1`, `Код: 1`. С таким кодом конвейер остановится. Ваш файл `zap-rules.tsv` в корне проекта при этом не изменился.

Остановите приложение и сохраните результат:

```
docker compose down -v
git add -A
git commit -m "lab5: access control, security headers, ZAP rules"
```

В отчёт: содержимое `zap-rules.tsv`, оба кода завершения.

Часть Б. Конвейер в GitHub Actions

Файл конвейера собирается из готовых блоков. В YAML отступы значимы: только пробелы, копируйте блоки целиком.

Шаг Б1. Опубликовать проект на GitHub

1. На https://github.com нажмите **+ → New repository**.
2. Имя `devsecops-lab`, видимость **Public**.
3. Галочки README, .gitignore, license **не ставьте** — репозиторий должен быть пустым.
4. **Create repository**.

Последняя проверка перед публикацией — нет ли секретов:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
```

Что увидите: `no leaks found`.

Отправьте код (подставьте свой логин):

```
git remote add origin https://github.com/<ваш_логин>/devsecops-lab.git
git push -u origin main
```

Git попросит войти в GitHub — откроется окно браузера. Если в терминале запрашивается пароль, нужен токен доступа: **Settings → Developer settings → Personal**

access tokens. Вводите его только в ответ на запрос. Не вставляйте токен в адрес репозитория и не сохраняйте в файлах проекта.

Что увидите: на странице репозитория ваши файлы и шесть коммитов.

В отчёт: ссылка на репозиторий.

Шаг Б2. Первое задание: тесты

```
mkdir -p .github/workflows
```

Создайте файл `.github/workflows/security.yml`:

```
name: security-pipeline

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  tests:
    name: Unit tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - name: Install dependencies
        run: python -m pip install -r requirements.txt -r requirements-dev.txt
      - name: Run tests
        run: pytest -q
```

Строка	Смысл
`on:`	когда запускать: отправка в `main`, любой pull request, вручную
`permissions: contents: read`	конвейеру разрешено только читать код
`runs-on: ubuntu-latest`	чистая виртуальная машина с Linux
`actions/checkout@v4`	получить код репозитория
`run:`	обычная команда терминала — та же, что на ноутбуке

```
git add -A
git commit -m "ci: add unit tests job"
git push
```

Откройте на GitHub вкладку **Actions**. Через минуту-две запуск станет зелёным. Откройте его → задание `Unit tests` → найдите в журнале `11 passed`.

Если запуск красный — откройте журнал упавшего шага. Чаще всего дело в отступах YAML.

В отчёт: снимок экрана зелёного запуска.

Шаг Б3. Три сканера

Добавьте в **конец** файла три задания. Слова `secrets:`, `sast:`, `sca:` должны стоять с тем же отступом, что и `tests:` (два пробела).

```
  secrets:
    name: Secrets (Gitleaks)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: Install Gitleaks
        run: |
          curl -sSfL https://github.com/gitleaks/gitleaks/releases/download/v8.2
4.2/gitleaks_8.24.2_linux_x64.tar.gz \
            | sudo tar -xz -C /usr/local/bin gitleaks
      - name: Scan Git history
        run: |
          mkdir -p reports
          gitleaks git --redact --no-banner --report-format json --report-path r
eports/gitleaks.json .
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: gitleaks-report
          path: reports/

  sast:
    name: SAST (Semgrep)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - name: Install Semgrep
        run: python -m pip install -r requirements-dev.txt
      - name: Run Semgrep
        run: |
          mkdir -p reports
          semgrep scan --config .semgrep.yml --metrics=off --error --json-output
```

```
 reports/semgrep.json .
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: semgrep-report
          path: reports/

  sca:
    name: SCA and SBOM (Trivy)
    runs-on: ubuntu-latest
    env:
      TRIVY_IMAGE: aquasec/trivy:0.76.2
    steps:
      - uses: actions/checkout@v4
      - name: Scan dependencies
        run: |
          mkdir -p reports && chmod 777 reports
          docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" fs --scanners vuln \
            --severity HIGH,CRITICAL --exit-code 1 /src
      - name: Check Dockerfile
        run: docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" config --exit-code 1
/src
      - name: Generate SBOM
        run: |
          docker run --rm -v "$PWD:/src" "$TRIVY_IMAGE" fs --format cyclonedx \
            --output /src/reports/sbom.cdx.json /src
      - name: Upload SBOM
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: sbom
          path: reports/
```

Найдите в блоках знакомое и запишите в таблицу для отчёта:

Задание	Команда сканера	Что делает проверку блокирующей	Из какой работы
`secrets`	`gitleaks git`	код 1 при находке	ЛР3
`sast`			
`sca`			

Три новые детали:

- `fetch-depth: 0` — получить **всю** историю. Без этого Gitleaks проверил бы один последний коммит;
- `if: always()` — сохранить отчёт, даже если проверка упала: именно тогда он нужнее всего;
- `.gitleaksignore` из ЛР3 учитывается автоматически.

```
git add -A
git commit -m "ci: add Gitleaks, Semgrep and Trivy jobs"
git push
```

Что увидите: четыре зелёных задания, выполняются одновременно. Внизу страницы запуска, в разделе **Artifacts**, три архива с отчётами. Скачайте `sbom` — внутри тот же CycloneDX, что в ЛР4.

В отчёт: таблица, снимок экрана запуска с разделом Artifacts.

Шаг Б4. DAST в конвейере

Добавьте в конец файла последнее задание:

```
  dast:
    name: Container and DAST (ZAP)
    runs-on: ubuntu-latest
    needs: [tests, secrets, sast, sca]
    steps:
      - uses: actions/checkout@v4
      - name: Start application
        run: docker compose up -d --build --wait
      - name: Smoke test
        run: curl --fail --silent http://127.0.0.1:5000/health
      - name: ZAP baseline scan
        run: |
          mkdir -p reports
          cp zap-rules.tsv reports/
          chmod -R 777 reports
          docker run --rm --network host -v "$PWD/reports:/zap/wrk/:rw" \
            ghcr.io/zaproxy/zaproxy:stable zap-baseline.py \
            -t http://127.0.0.1:5000 -c zap-rules.tsv -r zap-report.html -I
      - name: Stop application
        if: always()
        run: docker compose down -v
      - name: Upload ZAP report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: zap-report
          path: reports/
```

`needs: [tests, secrets, sast, sca]` — запускать только после успеха четырёх предыдущих. Незачем поднимать приложение и сканировать его, если в коде уже нашли секрет или инъекцию. Порядок «сначала быстрое и дешёвое, потом медленное» — общее правило конвейеров.

```
git add -A
git commit -m "ci: add container start and ZAP baseline"
git push
```

Что увидите: пять заданий; `dast` стартует после остальных. На странице запуска — схема заданий со стрелками. Всего 3–5 минут.

Откройте журнал `dast` → шаг `ZAP baseline scan` и найдите итоговую строку. Сравните с тем, что получили на ноутбуке в шаге А6.

В отчёт: снимок экрана схемы заданий, итоговая строка ZAP из журнала, полный текст `security.yml`.

Шаг Б5. Опыт: внести уязвимость

Конвейер, который ни разу не покраснел, ещё ничего не доказал. Сыграйте роль разработчика, который вернул уязвимую версию страницы комментариев.

```
git switch -c feature/rich-comments
cp ../secure-development-labs/vulnerable-app/frontend/templates/comments.html fr
ontend/templates/
git diff
```

Что увидите: вернулся фильтр `|safe` — тот самый XSS из ЛР1.

```
git add -A
git commit -m "Allow rich text in comments"
git push -u origin feature/rich-comments
```

Локальный хук коммит пропустил: он ищет секреты, а не XSS.

На GitHub появится кнопка **Compare & pull request** — создайте PR в `main`.

Что увидите через 1–2 минуты:

- `Unit tests` и `SAST (Semgrep)` — красные;
- `Container and DAST (ZAP)` — пропущено (skipped): сработало `needs`;
- внизу страницы PR — сообщение о непройденных проверках.

Откройте журнал `SAST (Semgrep)` — найдите правило и строку. Откройте `Unit tests` — найдите имя упавшего теста.

Вопрос для отчёта: уязвимость поймали две проверки. Какая из них сработала бы, не будь в проекте тестов? А если бы в `.semgrep.yml` не было правила про `safe`?

В отчёт: снимок экрана красного PR, фрагменты двух журналов, ответ.

Шаг Б6. Исправить и принять

Верните исправленную версию в ту же ветку:

```
cp ../secure-development-labs/solutions/lab2/frontend/templates/comments.html fr
ontend/templates/
git add -A
git commit -m "Revert unsafe filter"
git push
```

Что увидите: PR перепроверился сам и стал зелёным. Нажмите **Merge pull request**, затем:

```
git switch main
git pull
```

В отчёт: снимок экрана зелёного PR.

Шаг Б7. Сделать проверки обязательными

Сейчас красный PR всё ещё можно принять — кнопка лишь предупреждает. Закройте эту возможность.

На GitHub: **Settings → Rules → Rulesets → New branch ruleset**.

1. Имя `protect-main`, Enforcement status — **Active**.
2. Target branches → **Include default branch**.
3. Включите **Require a pull request before merging**.
4. Включите **Require status checks to pass** и добавьте пять проверок: `Unit tests`, `Secrets (Gitleaks)`, `SAST (Semgrep)`, `SCA and SBOM (Trivy)`, `Container and DAST (ZAP)`.
5. **Create**.

Теперь изменение попадает в `main` только через PR и только с зелёным конвейером.

В отчёт: снимок экрана правила.

Шаг Б8. Итог практикума

Вернитесь к плану из ЛР1 и заполните, как оказалось на самом деле:

№	Проблема	Чем найдена на самом деле	Исправлена?	Проверяется в CI
1	SQL-инъекция	Semgrep, вручную	да, ЛР2	`sast`, `tests`
2	XSS			
3	Чужой профиль без входа			
4	Отладочный режим			
5	Токен в коде			
6	Старые библиотеки			
7	Открыт порт базы данных			
8	Пароли открытым текстом			

Сравните с вашей гипотезой из ЛР1: где вы ошиблись?

Составьте список того, что осталось неисправленным (остаточные риски). Минимум: пароли открытым текстом, нет защиты от CSRF, ключ сессии по умолчанию, уязвимости базового образа без исправлений.

И ответьте на главный вопрос в 4–6 предложениях: **конвейер зелёный — можно ли выпускать приложение в эксплуатацию?**

В отчёт: таблица, остаточные риски, ответ.

Дополнительные задания

По желанию, оцениваются отдельно.

1. **Dependabot.** Добавьте `.github/dependabot.yml` для `pip` и `github-actions`, покажите созданный им PR.
2. **Сканирование образа в CI.** Добавьте в `dast` шаг `trivy image … --ignore-unfixed` с сохранением отчёта. Объясните выбор порога.
3. **Готовые правила Semgrep.** Добавьте в `sast` второй шаг с `--config p/default` и подавлением ложного срабатывания из ЛР2. Блокирующий он или информационный? Почему?

Отчёт

Часть А

1. Итог первого сканирования ZAP, таблица десяти предупреждений, два ответа.
2. Доказательство доступа к чужому профилю.
3. Результаты после исправления: `pytest`, код 401, 10 → 6, таблица решений.
4. Файл `zap-rules.tsv`, два кода завершения.

Часть Б

5. Ссылка на репозиторий.
6. Полный текст `security.yml`.
7. Таблица «задание — команда — что блокирует».
8. Снимки экрана: зелёный запуск с пятью заданиями и артефактами, красный PR, зелёный PR, правило защиты ветки.
9. Итоговая таблица, остаточные риски, ответ о выпуске.

Контрольные вопросы

1. Чем DAST отличается от SAST? Приведите по одному дефекту практикума, который видит только один из них.
2. Почему `FAIL: 0` в отчёте ZAP не доказывает безопасность?
3. Почему доступ к чужому профилю не нашёл ни один сканер?
4. Как GitHub Actions понимает, что шаг провален?
5. Зачем заданию `secrets` параметр `fetch-depth: 0`?
6. Что изменится, если убрать `needs` у задания `dast`?
7. Зачем `if: always()` у шага сохранения отчёта?
8. Почему в конвейере указаны точные версии сканеров?

9. Сканер в конвейере не смог скачать базу уязвимостей и завершился с ошибкой. Как это трактовать?

Оценивание

Критерий	Баллы
Часть А. Сканирование ZAP, разбор предупреждений	15
Проверка исправления, решения по остаткам	10
Файл правил и проверка порога	10
Часть Б. Публикация, задание с тестами	10
Три сканера, артефакты, таблица	15
DAST в конвейере с needs	10
Опыт с уязвимым изменением: красный и зелёный PR	15
Защита ветки	5
Итог практикума	10
Итого	**100**